Darren Lee
CSE13S

Week 5 Notes

4/26/21

Starting Small

The Heart of the Machine

- Computers do not execute the languages you use
- They compile them
- Should you trust the compiler?
- Reflections on Trusting Trust (Ken Thompson - Turing Award Lecture 1984)
- SolarWinds attack
- Supply chain attacks; insert malicious code into the compiler
- UMN Inserts bugs in Linux as an “experiment” (do not do this)

What is the Machine Really Doing?

- And why do I care?
- Performance
- Understanding how the machine works helps you understand performance problems
- Security
- Debugging with Sharp Edges
- Reverse engineering
- Constant time checks

Debugging with Sharp Edges

- Heisenbugs-appear and disappear because of timing or because of what machine the code is running on; C and other higher-level languages enables portable code
- The high level doesn’t match the low level
- Timing problems

Constant Time Checks

- A byte by byte comparison of a password or other key
- Used to thwart an attacker
- An invalid answer should take as long as a valid one

What is Wrong with this Picture?

```
for (i = 0; i < len(user); i++)
    if (user_input[i] != key[i])
        exit(1);
// user_input matched key so continue from here
transfer_money_from_savings(alice, bob);
```

A Better Answer

```
for (i = 0; i < len(user); i++)
    if (user_input[i] != key[i])
        bad++;
    else
        good++;
if (bad > 0)
    exit(1);
// user_input matched key so continue from here
transfer_money_from_savings(alice, bob);
```

```
<- good is never used
   the compiler can remove it!
```

How do you check the compiler's work?

- Testing
  - Run 1000s of tests against the code and time them
- Read the machine code
  - For a small routine like this it is not that difficult

Starting Really Small

- A modern server processor has 100s of instructions
- A common embedded processor has 30
- Let's look at a worked example

A worked example

- AVR instruction set in 2 pages
- https://wokwi.com/arduino/projects/288885987581362700

Sometimes you have to debug problems in the machine code
If you learn the instruction set, you can look at things when they're broken or check constant time checks and tell if its correct or not

Questions?

- Simulator shamelessly stolen from this course on AVR assembly by Uri Shaked
- https://hackaday.com/tag/avr/
- https://hackaday.io/urishaked
- https://docs.google.com/document/d/1YAoqje3fdXm8g1e6mtvFzT3dm_MDSvBTEQl7OoT5Fbo/edit#

4/28/21
Recursion
A recursive function is defined in terms of itself
Can use it to sum the first k natural numbers

- $\sum k = k + \sum(k-1)$ and
- $\sum 1 = 1$, or

- We could loop through and add them up
- Or we can use Gauss's formula
- The first of these is recursive, since it is defined in terms of itself

```
int sum_1(int k) {
  if (k > 1) {
    return k + sum_1(k - 1);
  } else {
    return 1;
  }
}

int sum_2(int k) {
  int s = 1;
  for (int i = 2; i <= k; i += 1) {
    s += i;
  }
  return s;
}

int sum_3(int k) {
  return (k * (k + 1)) / 2;
}
```

Fibonacci numbers

- In this case, both algorithms compute the same value
- The first algorithm is recursive and runs in $O(2^n)$ time
- The second algorithm is iterative and runs in $O(n)$ time

```
int fib_1(int k) {
  if (k == 0 || k == 1) {
    return k;
  } else {
    return fib_1(k - 1) + fib_1(k - 2);
  }
}

int fib_2(int k) {
  int a = 0, b = 1, s = 0;
  if (k < 2) {
    return k;
  }
  for (int i = 2; i <= k; i += 1) {
    s = a + b;
    a = b;
    b = s;
  }
  return s;
}
```

recursive functions don't necessarily have the same complexity as loops
as seen in this case
typically if you want to get the time complexity of a recursive function, you have to analyze the "recurrence relation"

Are recursive algorithms inherently inefficient?

- No
- A function call requires creating a stack frame, and that takes time and space
- All tail recursive functions can be rewritten as iteration (often by the compiler)
- LIFO stack

So, when do I use recursion

- When it makes sense
- Do not use it when it does not make the code clearer

```
int list[] = { 9, 3, 2, 7, 5, 1, 0, 4, 6, 8 };

int min_1(int *l, int k) {
  if (k == 1) {
    return *l;
  } else {
    int m = *l++;
    int n = min_1(l, k - 1);
    return m < n ? m : n;
  }
}

int min_2(int *l, int k) {
  int m = *l++;
  for (int i = 1; i < k; i += 1) {
    if (*l < m) {
      m = *l++;
    } else {
      l += 1;
    }
  }
```

Tail recursion-compiler makes sure that the last thing you do is on the right hand side

Binary search

- We can search an ordered array in O(log(n)) time
- If the list is empty, then it is not there
- If the key is smaller, look in the left half
- If it is larger then look in the right half

```
int search(int key, int a[], int left, int right) {
  if (left > right) {
    return -1; // Not found
  } else {
    int middle = (left + right) / 2;
    if (key < a[middle]) {
      return search(key, a, left, middle - 1);
    } else if (k > a[middle]) {
      return search(key, a, middle + 1, right);
    } else {
      return middle;
    }
  }
}
```

You can also do this iteratively

How fast can we search

- Binary search is as fast as we can go
- What about slicing it into three parts? Or 10?
- It's still O(log(n)), just with a different constant
- Remember: binary search requires that the array be sorted

String Table

- Suppose that I want to efficiently store strings so that I only have a single copy of each one
- Why would I want to do that
- Compiler have to keep track of a lot of strings (variable names, for example)

```
#ifndef _STR_TREE_
#define _STR_TREE_

#include <strings.h>

typedef struct str_tree {
  char *entry;
  struct str_tree *left, *right;
} str_tree;

extern str_tree *str_root;

extern char *str_find(char *);
extern char *str_insert(char *);
#endif
```

new_node()

- Allocate a node
- Set the children to NULL
- Make room for the string
- Copy the string

```
static str_tree *new_node(char *key) {
  str_tree *node;

  node = (str_tree *)malloc(sizeof(str_tree));

  node->left  = (str_tree *)0; // New nodes have no
  node->right = (str_tree *)0; // children.

  node->entry = (char *)malloc(strlen(key) + 1);

  strcpy(node->entry, key);

  return node;
}
```

Bad!

-

strcpy() doesn't check bounds, so it's bad

str_find()

- Start at the root
- Are we done?
- What does strcmp() say?
- Go left?
- Go right?
- There it is!
- Oops, did not find it

```
char *str_find(char *key) {
  str_tree *chase;

  int direction;

  chase = str_root;

  while (chase != (str_tree *)0) {

    direction = strcmp(key, chase->entry);

    if (direction < 0) {
      chase = chase->left;
    } else if (direction > 0) {
      chase = chase->right;
    } else {
      return chase->entry;
    }
  }

  return (char *)0;
}
```

A recursive alternative

- If we've reached the leaves, then it isn't there
- What does strcmp() say?
- Recurse left?
- Recurse right?
- There it is

```
char *str_find(str_tree *root, char *key) {

  int direction;

  if (root != (char *)0) {
    direction = strcmp(key, chase->entry);

    if (direction < 0) {
      return str_find(root->left, key);
    } else if (direction > 0) {
      return str_find(root->right, key);
    } else {
      return root->key;
    }
  }
  return (char *)0;
}
```

String trees were for when you needed to store strings when writing a compiler

Recursion is natural for search

- We use recursion for searching by dividing the space up
- Places we have searched
- And places we have yet to search
- If we get stuck, we can back up and try a different path
- For example, we want to find the shortest path from E2 to Betty Burgers

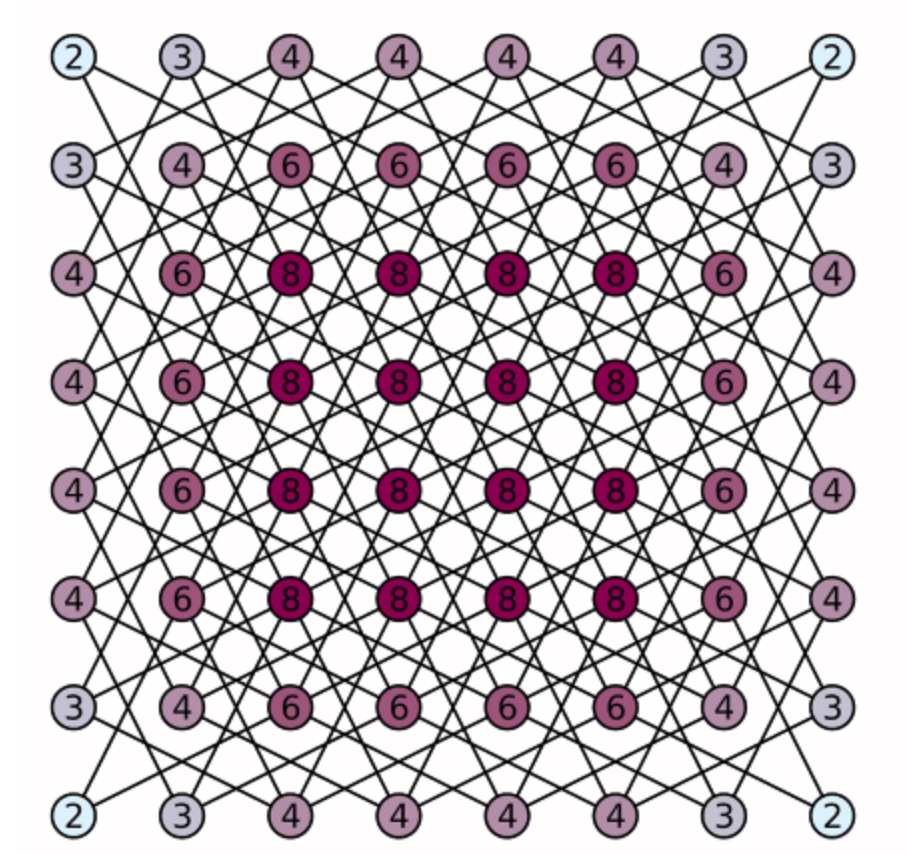
The Knight's Tour

- Can a knight visit all of the squares on a chess board exactly once?
- This is a form of a Hamiltonian path problem

```
bool attempt(uint32_t i, uint32_t x, uint32_t y) {
  uint32_t k = 0;
  bool quit;
  do {
    uint32_t u, v;
    count += 1;
    quit = false;
    u = x + a[k];
    v = y + b[k];
    if ((u < n && u >= 0) && (v < n && v >= 0)) { // On the board?
      if (chess_board[u][v] == 0) {               // Vacant?
        chess_board[u][v] = i;                    // Place knight
        if (i < n * n) {                          // Done?
          quit = attempt(i + 1, u, v);
          if (quit == false) {
            chess_board[u][v] = 0; // Back up and try again
          }
        } else {
          quit = true;
        }
      }
    }
    k += 1;
  } while (quit == false && k < 8);
  return quit;
}
```

How many possible moves?

There are 26,534,728,821,064 directed tours on an 8 × 8 chess board.




```
darrell@hilbert Examples % clang -DDEBUG=1 -o knight knight.c
darrell@hilbert Examples % ./knight -n 5
        1       6       15      10      21
        14      9       20      5       16
        19      2       7       22      11
        8       13      24      17      4
        25      18      3       12      23
Solution required 70624 tries.
darrell@hilbert Examples % ./knight -n 8
        1       60      39      34      31      18      9       64
        38      35      32      61      10      63      30      17
        59      2       37      40      33      28      19      8
        36      49      42      27      62      11      16      29
        43      58      3       50      41      24      7       20
        48      51      46      55      26      21      12      15
        57      44      53      4       23      14      25      6
        52      47      56      45      54      5       22      13
Solution required 66005601 tries.
darrell@hilbert Examples % ./knight -n 10
Try 1000000000
Try 2000000000 ←——— Perhaps not!
Try 3000000000
^C
```

Eight Queens

- Can we place eight queens on a chess board so that non threatens the others
- There are 92 solutions for an 8x8 chess board
- Larger boards have more solutions
- What is the expected runtime of this problem?

Print the Board

Solution due to Niklaus Wirth & Deepank Pant

```
void printSolution(int board[N][N]) {
  for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
      printf("%c ", board[i][j] ? 'Q' : '-');
    }
    printf("\n");
  }
}
```

Is this position safe?

```
bool isSafe(int board[N][N], int row, int col) {
  int i, j;
  // Rows
  for (i = 0; i < col; i++) {
    if (board[row][i]) { return false; }
  }
  // Left diagonal
  for (i = row, j = col; i >= 0 && j >= 0; i -= 1, j -= 1) {
    if (board[i][j]) { return false; }
  }
  // Right diagonal
  for (i = row, j = col; j >= 0 && i < N; i += 1, j -= 1) {
    if (board[i][j]) { return false; }
  }
  return true;
}
```

Search for the solution

```
bool solveNQP(int board[N][N], int col) {
  if (col >= N) { return true; }
  // Placing the queen in each row
  for (int i = 0; i < N; i++) {
    // Did that work?
    if (isSafe(board, i, col)) {
      // OK, do it!
      board[i][col] = 1;
      // Recurse to place the others
      if (solveNQP(board, col + 1)) {
        return true;
      }
      board[i][col] = 0; // Did not work back up!
    }
  }
  // No solution found!
  return false;
}
```

Recursion does not really exist in computers, the computers use the stack to manage recursion

Summary

- Recursion is natural
- It is good for search problems
- It is not inherently inefficient
- Like all tools and techniques, use it where it makes sense

Graphs

- Components are nodes, edges, and weights
- Can be directed if edges have a direction
- Edges may have no direction (or both directions), n1<->n2, and we call that an undirected graph
- The edges may have weights, which represent capacity, strength, or cost

Representing a graph

- Adjacency matrix
    - Nxn matrix
    - binary :edges present or absent
    - Weighted: n=/=0

Adjacency matrix

- A non-zero entry in $M_{i,j}$ means there is an edge n_i->n_j
- A matrix that is symmetric around the diagonal represents an undirected graph
- The entry can specify not only the existence of an edge, but also its weight
- Requires $O(n^2)$ space
    - Sparse matrix techniques can improve it

- Adjacency list
    - Each node is represented as an entry in a column vector
        - Each entry is the head of a linked list
    - The list elements contain:
        - The destination node, and
        - The weight of the edge
    - Why would you prefer this over an adjacency matrix?
        - An adjacency matrix is $O(n^2)$ space
        - An adjacency list will be more space efficient for sparse graphs

Trees
- Trees are a form of acyclic graph
    - Acyclic means that if you follow the edges, there are no loops (cycles)
- You will often hear the term DAG, which stand for: Directed Acyclic Graph

Basic Graph Algorithms
- Depth-first search
    - Based on recursion
    - Uses a stack
- Breadth first search
    - Based on a queue

Graph problems
- Shortest path
- Hamiltonian path
    - Traveling salesman problem

Social networks

Network routing
- The Internet (originally called ARPAnet) is a graph
    - The nodes are computers (one reason that we call computers nodes)
    - The edges are the connections
- Routers are nodes with many edges

4/30/21

Pointers and dynamic memory
- A pointer is variable that holds a memory address
    - The variable points to the location of an object in memory
- Not all pointers contain an address
    - Pointers that don't' contain an address are set to the NULL pointer
    - Value of the NULL pointer is 0
    - NULL is a macro for either
        - ((void *) 0)
        - 0
        - 0L

- When you don't need a pointer any more, set it to NULL

Memory addresses

- Memory is stored in registers that can be accessed by a specific number (address)
- Usually, each byte has a unique address
- Bytes are grouped into words (timeshare uses word size 4)

Pointers and addresses

- Pointers are said to point at the address they are assigned
- Can assign a pointer the address of a variable using the address-of operator (&)
- Multiple pointers can point to the same address

```c
#include <stdio.h>

int main(void) {
  int a = 42;
  int *ptr_a = &a;
  int *ptr_b = &a;

  // Two pointers can point to the same address.
  printf("The address of pointer A is %p\n", ptr_a);
  printf("The address of pointer B is %p\n", ptr_b);
  return 0;
}
```

Address	00	04	08	0C
A000_7FA0	00000000	00000000	00000000	00000000
A000_7FB0	00000000	00000000	00000000	00000000
A000_7FC0	00000000	00000000	A0007FD0	A0007FD0
A000_7FD0	0000002A	00000000	00000000	00000000
A000_7FE0	00000000	00000000	00000000	00000000

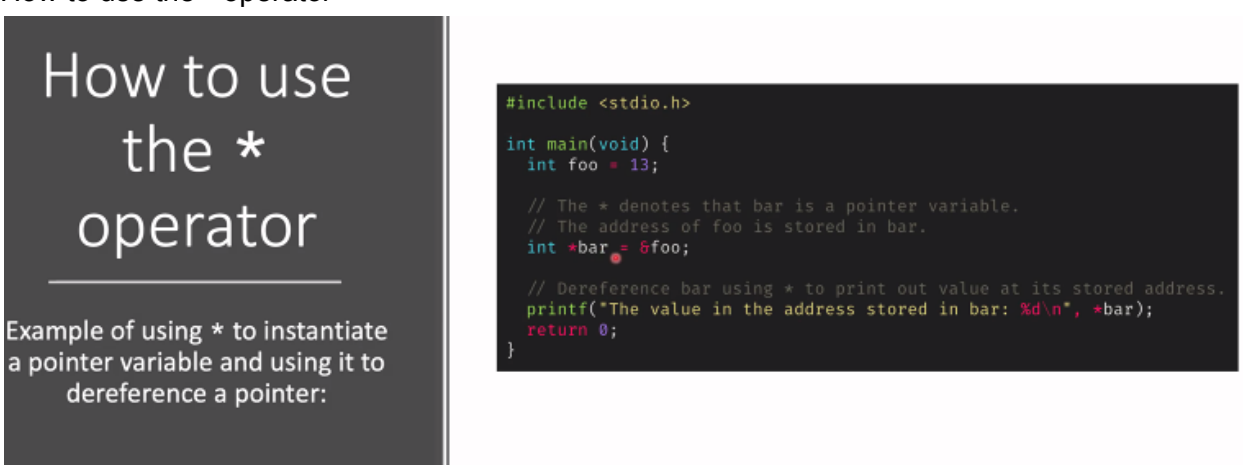
How to use the & operator

- Gets the address

Dereferencing a pointer

- The objects a pointer points to can be accessed through dereference, or indirection
- A pointer can be dereferenced using the dereferencing operator (*)

How to use the * operator



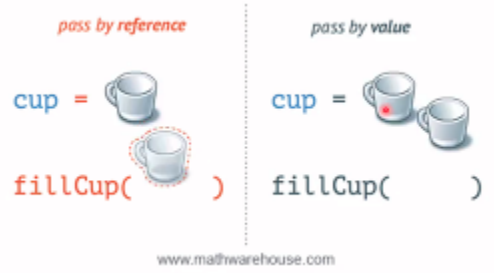
Benefits of pointers

- Can be used when passing actual values is difficult

- Can “return” more than one value from a function
- Building dynamic data structures
- Useful for passing large data structures around
    - Pointers are efficient for this since copies of data structures don’t need to be pushed

Passing by value vs. passing by reference

- Passing by value duplicates passed values onto stack
- Passing by reference duplicates a pointer onto the stack



Passing by reference

- Allows returning multiple values
- Allows passing large amounts of data quickly
    - You’re not copying the data, just telling where it is stored

Pointer arithmetic

- Since pointer inC are just an addresses, numeric values you can perform arithmetic on them
- ++: increments to next address (increment by 4 bytes if we are using 32-bit integers)
- --: decrements to previous address
- +: can only add a numeric value to a pointer (no pointer and pointer addition)
- -: if a pointer is subtracted from another pointer, the distance between both addresses is calculated
- Pointers can also be compared using relational operators (e.g. ==,<=,<,etc.)

- Can offset a pointer by adding/subtracting an integer
- Can get the number of elements between two pointers by getting their difference
- Cannot sum, divide, or multiply two pointers

-
```c
#include <stdio.h>

int main(void) {
  char *str = "Hello World!";
  printf("str[0]: %c\n", *str);
  printf("str[1]: %c\n", *(str + 1));
  return 0;
}
```

Arrays and pointer arithmetic

- For int arr[i]:
  - arr[i]==*(a+i)
  - If arr starts at address 1000. Then arr[i] is at 1000+(i*4)
    - An int is 4 bytes

But does it make sense?

- Adding an int to a pointer

Pointer arithmetic example: iterating over an array

```c
int main(void) {
  int fib[] = { 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 };

  int *fib_ptr = fib;
  while (fib_ptr < (fib + 10)) {
    *(fib_ptr)++;    // Increment array element.
    fib_ptr++;       // Increment pointer.
  }

  return 0;
}
```

fib_ptr

Address	00	04	08	0C
A000_7FA0	00000000	00000000	00000000	00000000
A000_7FB0	00000000	00000000	A0007FBC	00000001
A000_7FC0	00000002	00000002	00000003	00000004
A000_7FD0	00000005	00000008	0000000D	00000000
A000_7FE0	00000022	00000000	00000000	9D000390

Don't write code like this, use brackets instead

Pointers and arrays

- Array subscripting can also be done with pointers
  - Assuming some array int arr[10]:
    - Arr[i] i s equivalent to *(arr+i), weather 0<=i<10
- Arrays can always be written using pointers
  - Declaring an array in a function allocates it on the stack
  - A global arrays in the data area
  - Dynamically declaring an array (to get a pointer) allocates it on the heap

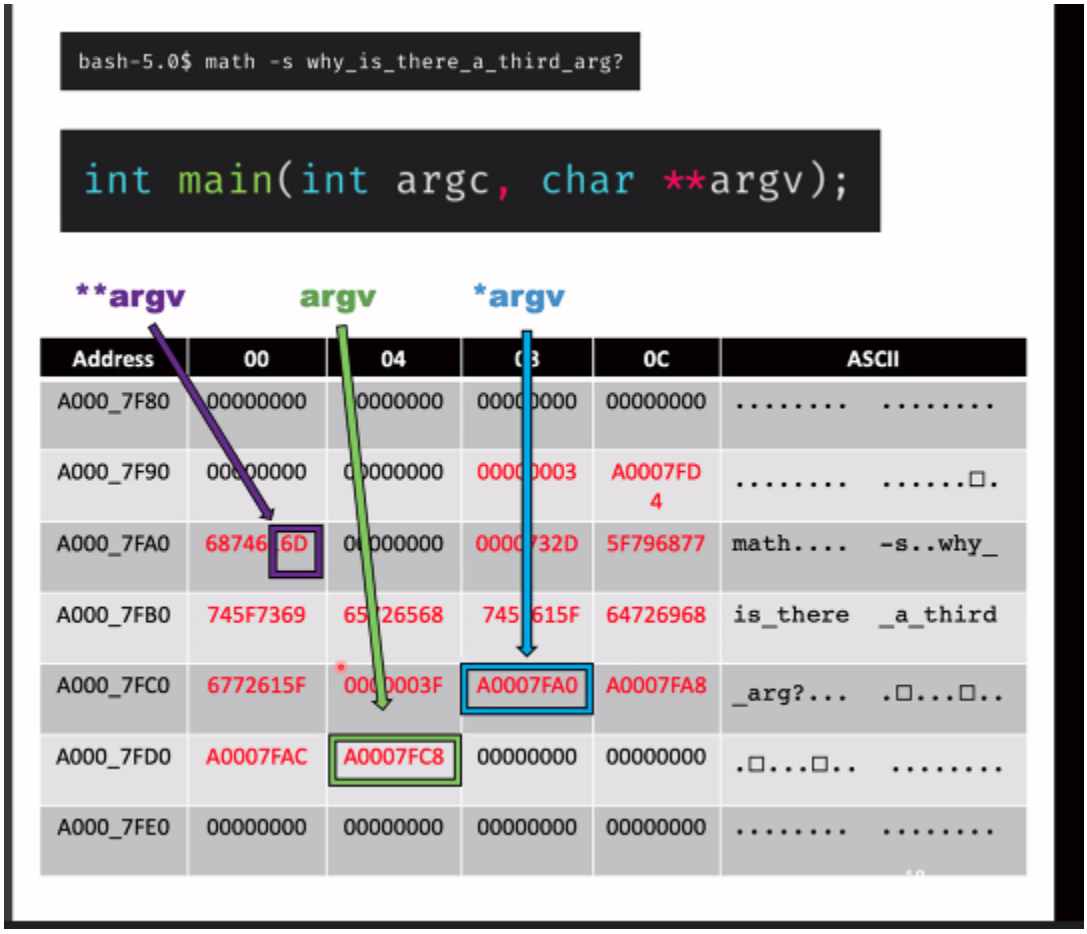
Strings as arrays

- In C, strings are handled as arrays
  - With some special syntax, for convenience
- A string is a pointer to an array of chars
- Strings can be indexed, passed by reference, etc

Pointers to pointers

- Pointer can point to other pointers
  - Or to pointers to other pointers, or to pointers to pointers to pointers, etc
- Can be used to pass arrays of arrays, such as a list of strings

- For example, char **argv



Multidimensional arrays

- In some cases, arrays of arrays support a more compact data structure
  - If data is of same type, and each sub-array is of same length, a multidimensional array is possible
  - Can be of any dimensionality
  - Note: these grow in size very quickly

Function pointers

- Points to executable code in memory instead of a data value
- Dereferencing a function pointer yields the referenced function
- Notice how there are parentheses around the function pointer
  - Without them, the function pointer would be parsed as a function declaration

```c
#include <stdio.h>

void increment(int *a) {
  *a += 1;
  return;
}

int main(void) {
  // Make func_ptr point at increment().
  void (*func_ptr)(int *) = increment;

  int x = 5;

  // Increment x by invoking increment().
  func_ptr(&x);
  return 0;
}
```

Function pointer example

```c
#include <stdio.h>
#include <math.h>

//
// Numerically computes the derivative of a function f about the point x0.
// Takes a double as an argument and returns a double.
//
double diff(double (*f)(double), double x0) {
  double epsilon = 0.00001;
  double delta_x = f(x0 + epsilon) - f(x0 - epsilon);
  return delta_x / (2 * epsilon);
}

int main(void) {
  for (double x = 0.0; x < (2 * M_PI); x += (M_PI / 16)) {
    printf("x = %6.3f, cos(x) = %6.3f, d/dx(cos(x)) = %6.3f\n"
      i, cos(x), diff(sin, x));
  }

  return 0;
}
```

Function tables (jump tables)

```c
#include <stdio.h>

enum { INCREMENT, DECREMENT };

void increment(int *a) {
  *a += 1;
  return;
}

void decrement(int *a) {
  *a -= 1;
  return;
}

int main(void) {
  // func_table is an array of function pointers.
  // func_table[0] is increment().
  // func_table[1] is decrement().
  void (*func_table[])(int *) = { INCREMENT, DECREMENT };

  int x = 5;

  func_table[INCREMENT](&x);  // Increments x to 6.
  func_table[DECREMENT](&x);  // Decrements x back to 5.
  return 0;
}
```

Dynamic memory allocation

- Memory that is allocated at runtime
- It is allocated from a region of memory that we call the heap

- The act of allocating memory for variables on the heap (or called free store) during program run-time
- Quite different than compile time (static) allocation:
    - Compile time allocation (CTA) means memory for named variables is allocated by the compiler at compile time
    - Dynamic memory allocation (DMA)

- Stack space is limited
- Sometimes we want variables to last beyond the lifetime of its current scope
    - Variables on the stack don't last beyond its current scope
    - Variables dynamically allocated on the heap can be accessed beyond the current scope
- We don't always know exactly how much memory is needed to run a program
    - Solution: dynamically allocate memory when it's needed, however much is needed

- There are 3 standard C library functions to allocate memory, all of which are defined under stdlib.h
    - malloc(), calloc(), realloc()
    - Should free() things that are allocated with these functions to prevent memory leaks

The Heap

- A large region of unmanaged, anonymous memory
- Only limitations are your computer's physical limitations
- Slower to read from/write due to the need for pointers
- Variables using heap memory can be accessed globally with access to the pointer
    - A benefit of using pointers; much easier to pass around pointers for large data structures
- Possible memory fragmentation can occur over time as blocks of memory are allocated and deallocated

malloc()

- Defined as:
    - Void* malloc (size_t size)
- returns a pointer to [size] bytes of uninitialized memory allocated on the heap
- The memory allocated by malloc() may contain junk data
- What happens when zero is passed as the specified number of bytes to allocate is implementation defined (avoid doing this)
- Doesn't check for overflow of [size] if it's the result of an arithmetic operation

Example of malloc()

```c
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>

//
// Allocates memory for an array of 10 ints.
// Sets each array index to the value of the index.
//
int main(void) {
  int *arr = malloc(10 * sizeof(int));
  assert(arr);

  for (int i = 0; i < 10; ++i) {
    arr[i] = i;
  }

  return 0;
}
```

calloc()

- Defined as:
    - Void* calloc(size)t nmemb, size_t size)
- [nmemb] denotes the number of objects and [size] the size of each object
- Returns a pointer to [nmemb]

Example of calloc()

```c
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>

//
// Functions like previous malloc() example.
// Takes advantage of zeroed-out array to reach same result.
//
int main(void) {
  int *arr = calloc(10, sizeof(int));
  assert(arr);

  for (int i = 0; i < 10; ++i) {
    arr[i] += i;
  }

  return 0;
}
```

Dynamic memory allocation example

- A dynamic nxn matrix
- Example of dynamically allocating an nxn matrix
- The destructor matrix_delete() iterates and frees the allocated memory for each pointer

- ```c
  #include <assert.h>
  #include <stdio.h>
  #include <stdlib.h>

  int **matrix_create(int n) {
    int **m = (int **)calloc(n, sizeof(int *));
    assert(m);

    for (int i = 0; i < n; ++i) {
      m[i] = (int *)calloc(n, sizeof(int));
      assert(m[i]);
    }

    return m;
  }

  void matrix_delete(int **m, int n) {
    for (int i = 0; i < n; ++i) {
      free(m[i]);
      m[i] = NULL;
    }

    free(m);
    m = NULL;
    return;
  }

  int main(void) {
    int n = 10;
    int **m = matrix_create(n);

    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        printf("%d ", m[i][j]);
      }

      printf("\n");
    }

    matrix_delete(m);
    return 0;
  }
  ```

Dynamic matrix:
main()
  ```

```c
int main(void) {
  int n = 10;
  int **m = matrix_create(n);

  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      printf("%d ", m[i][j]);
    }

    printf("\n");
  }

  matrix_delete(m);
  return 0;
}
```

matrix_create()

```c
int **matrix_create(int n) {
  int **m = (int **)calloc(n, sizeof(int *));
  assert(m);

  for (int i = 0; i < n; ++i) {
    m[i] = (int *)calloc(n, sizeof(int));
    assert(m[i]);
  }

  return m;
}
```

Matrix_delete

```c
void matrix_delete(int **m, int n) {
  for (int i = 0; i < n; ++i) {
    free(m[i]);
    m[i] = NULL;
  }

  free(m);
  m = NULL;
  return;
}
```

Empty the interior part of the container. Then get rid of the container

realloc()

- Defined as:
  - Void *realloc(void *ptr,size_t size)
- Reallocates [ptr] to newly point at [size] bytes of allocated memory on the heap
  - This is not strictly true; what realloc() actually does is deallocate the old object pointed to by [ptr] and return a pointer to [size] bytes of allocated but uninitialized memory

A dynamically growing stack

```c
#ifndef __STACK_H__
#define __STACK_H__

#include <inttypes.h>
#include <stdbool.h>

#define MIN_STACK 16
#define INVALID   0xdeadd00d

typedef uint32_t item;

typedef struct Stack {
  uint32_t size;
  uint32_t top;
  item *entries;
} Stack;

Stack *stack_create(void);

void stack_delete(Stack *);

item stack_pop(Stack *);

void stack_push(Stack *);

bool stack_empty(Stack *);

#endif
```

```c
Stack *stack_create(void) {
  Stack *s = (Stack *)malloc(sizeof(Stack));
  assert(s);
  s->size = MIN_STACK;
  s->top = 0;
  s->entries = (item *)calloc(MIN_STACK, sizeof(i
  return s;
}
```

Dynamic Stack: `stack_create()`

```
void stack_push(Stack *s, item i) {
  if (s->top == s->size) {
    s->size *= 2;
    s->entries = (item *)realloc(s->entries, s->size * siz
  }

  s->entries[s->top] = i;
  s->top += 1;
  return;
}
```

Dynamic Stack: `stack_push()`

free()

- If we allocate memory, we must also be able to deallocate (or free) memory
- Another standard C library functions
- Defined as void free(void *ptr)
- Deallocates the memory space pointed to by [otr]
- Memory leaks occur if allocated memory isn't freed
- Segmentation faults/core dumps can occur if a program tries to access memory locations that it isn't allowed to access
- Pointer that have been freed should be set to NULL to mitigate use-after-free vulnerabilities

Example of free()

```c
#include <assert.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
  int **arr = (int **)calloc(10, sizeof(int *));
  assert(arr);

  for (int i = 0; i < 10; ++i) {
    arr[i] = (int *)calloc(10, sizeof(int));
    assert(arr[i]);
  }

  for (int i = 0; i < 10; ++i) {
    free(arr[i]);
    arr[i] = NULL;
  }

  free(arr);
  arr = NULL;
  return 0;
}
```

© 2020 Darrell Long

Detecting memory leaks and segmentation faults

- Memory leaks and segmentation faults, as well as other errores, can be detected using gdb, infer, or valgrind
    - These tools can be found on the UCSC Unix timeshare
    - Should be used in tandem when debugging

Valgrind

- A collection of dynamic analysis tools
- Usually used for its memcheck tool, which can detect
    - Use of uninitialized memory
    - reading/writing memory after it has been freed
    - reading/writing off the end of allocated blocks of memory
    - reading/writing inappropriate reas on the stack
    - Memory leaks - where pointers to malloc'd blocks are lost forever
- Useful command line flags/options for valgrind
    - --leak-check=full

Static vs dynamic analyzers

- Static analyzers, like infer, operate by analyzing the source code for a program before it's run
    - Cod is compared against a set (or multiplied sets) of coding rules for bugs
    - Only surface level; can't check if a function behaves completely correctly